

DL Midsem Notes — Section 7 Supplement

Adam Optimizer, In Detail

1 The Problem Adam Is Solving

Two separate weaknesses, two separate fixes, Adam is just both fixes stapled together:

- Plain SGD's direction is noisy (single-sample or mini-batch gradient) and it uses one global η for every parameter, even though some parameters need bigger steps and some need smaller ones. **Momentum** fixes the noise by averaging gradients over time.
- Different parameters can have very different gradient magnitudes/frequencies (e.g. an embedding row updated rarely vs. a weight updated every step). **RMSProp** fixes this by giving each parameter its own adaptive step size based on how large its recent gradients have been.

Adam = momentum's first moment + RMSProp's second moment, run together, each with bias correction.

2 First Moment m_t — the Direction

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad \beta_1 \approx 0.9$$

This is an exponential moving average (EMA) of the gradient itself. Expand the recursion:

$$m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i$$

Recent gradients get weight $(1 - \beta_1)$, gradients k steps ago get weight $(1 - \beta_1)\beta_1^k$ — exponentially decaying, so m_t behaves like "the recent trend," roughly averaging over the last $\sim 1/(1 - \beta_1) = 10$ steps for $\beta_1 = 0.9$. **Effect:** if gradients keep pointing roughly the same way, m_t builds up and accelerates you there; if they oscillate (flip sign step to step, e.g. across a narrow valley), the average partially cancels out and damps the oscillation.

3 Second Moment v_t — the Scale

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \beta_2 \approx 0.999$$

Same idea, but averaging *squared* gradients — this tracks the recent *magnitude* of the gradient (regardless of sign), i.e. how volatile/large this parameter's gradient typically is. β_2 is set much closer to 1 than β_1 (window $\sim 1/(1 - \beta_2) = 1000$ steps) because you want the magnitude estimate to be a stable, slowly-changing quantity — reacting to every recent gradient's magnitude too fast would make the effective step size jump around instead of adapting smoothly.

What v_t is used for: dividing the update by $\sqrt{v_t}$ rescales the step per-parameter: a parameter whose recent gradients have been large gets a smaller effective step (it's already changing a lot, no need to push harder); a parameter whose recent gradients have been small/rare gets a relatively larger effective step (it needs help catching up).

4 Why Bias Correction Is Necessary

$m_0 = v_0 = 0$. From the expansion above, if every past gradient happened to equal the current true gradient g , then

$$m_t = (1 - \beta_1)g \sum_{i=1}^t \beta_1^{t-i} = g(1 - \beta_1^t)$$

So m_t systematically *underestimates* g by a factor $(1 - \beta_1^t)$, worst right at the start (small t) where β_1^t is still close to 1. Dividing by exactly that factor removes the bias:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

As $t \rightarrow \infty$, $\beta_1^t, \beta_2^t \rightarrow 0$, so both correction factors $\rightarrow 1$ — the correction matters only in early steps and fades out on its own (this is exactly Q4 from section 7’s PYQ list).

5 Putting It Together

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

$\epsilon \sim 10^{-8}$ only prevents division by zero when $\hat{v}_t \approx 0$; it has no other role. Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ work well across most problems, which is a big part of Adam’s popularity — little to no per-problem tuning needed, unlike plain SGD where η often needs careful hand-tuning per model.

5.1 A Property Worth Noticing: Self-Bounded Step Size

When gradients are consistently signed (as in the worked example below), $\hat{m}_t/\sqrt{\hat{v}_t}$ stays close to ± 1 in magnitude, so each update is roughly $|\Delta w| \approx \eta$ — the step size in that regime is controlled almost entirely by η , not by how large the raw gradient happens to be. This is part of why Adam is comparatively forgiving of the raw gradient’s scale (unlike vanilla SGD, where a rescaled loss directly rescales every step).

6 Worked Numerical Example

Single scalar parameter $w_0 = 1.0$, $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Gradients observed at each step: $g_1 = 0.8$, $g_2 = 0.5$, $g_3 = 0.6$.

Step 1.

$$m_1 = 0.9(0) + 0.1(0.8) = 0.08, \quad v_1 = 0.999(0) + 0.001(0.64) = 0.00064$$

$$\hat{m}_1 = \frac{0.08}{1 - 0.9^1} = \frac{0.08}{0.1} = 0.8, \quad \hat{v}_1 = \frac{0.00064}{1 - 0.999^1} = \frac{0.00064}{0.001} = 0.64$$

Notice: since $m_0 = v_0 = 0$, at $t = 1$ the correction exactly cancels the averaging, giving $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$ precisely — bias correction fully undoes the zero-initialization bias right from the first step.

$$w_1 = 1.0 - 0.001 \cdot \frac{0.8}{\sqrt{0.64} + \epsilon} = 1.0 - 0.001 \cdot \frac{0.8}{0.8} = 1.0 - 0.001 = \boxed{0.999000}$$

Step 2.

$$m_2 = 0.9(0.08) + 0.1(0.5) = 0.122, \quad v_2 = 0.999(0.00064) + 0.001(0.25) = 0.00088936$$

$$\hat{m}_2 = \frac{0.122}{1 - 0.81} = 0.6421, \quad \hat{v}_2 = \frac{0.00088936}{1 - 0.998001} = 0.44490, \quad \sqrt{\hat{v}_2} = 0.6670$$

$$w_2 = 0.999000 - 0.001 \cdot \frac{0.6421}{0.6670} = 0.999000 - 0.000963 = \boxed{0.998037}$$

Step 3.

$$m_3 = 0.9(0.122) + 0.1(0.6) = 0.1698, \quad v_3 = 0.999(0.00088936) + 0.001(0.36) = 0.00124847$$

$$\hat{m}_3 = \frac{0.1698}{1 - 0.729} = 0.6266, \quad \hat{v}_3 = \frac{0.00124847}{1 - 0.997003} = 0.41657, \quad \sqrt{\hat{v}_3} = 0.6454$$

$$w_3 = 0.998037 - 0.001 \cdot \frac{0.6266}{0.6454} = 0.998037 - 0.000971 = \boxed{0.997066}$$

Notice each step moves w by very close to $\eta = 0.001$, even though the raw gradients ranged from 0.5 to 0.8 — exactly the self-bounded-step behaviour described above, because all three gradients shared the same sign.

7 Adam vs. the Others, at a Glance

	Uses momentum?	Per-parameter adaptive scale?	Bias-corrected?
SGD	No	No	—
Momentum	Yes	No	—
RMSProp	No	Yes	No
Adam	Yes	Yes	Yes